Literature Review: CPU Scheduling and Resource Management, 2023–2026

Project: ThreadHaul-Daemon (logistics dispatch server resource monitor and priority throttler)

Scope: Peer-reviewed work published 2023–2026 in IEEE, ACM, Elsevier, Springer, MDPI and Nature-portfolio venues on OS-level and server-level CPU scheduling, colocation of latency-critical and batch work, and load prediction.

Purpose: Find gaps in how present-day schedulers protect latency-critical processes, and justify each major design decision in ThreadHaul-Daemon.

---

1. Method

This is an informal, structured scan, as the spec asks for. It is not a full systematic review.

1 **Search.** The Crossref metadata API was queried with bibliographic searches limited to 2023-01-01 – 2026-12-31 and to journal articles or proceedings papers. Query themes:

- colocation of latency-critical and best-effort/batch work;
- interference-aware scheduling (memory, I/O, CPU contention);
- workload/CPU-load prediction for proactive allocation;
- surveys of CPU / OS scheduling.

A general web search was used alongside it to find well-known venue papers (SOSP, IEEE/ACM ToN).

1 **Screening.** A paper was kept if:

- its venue is one of IEEE, ACM, Elsevier, Springer, Wiley, MDPI or Nature;
- it was published 2023–2026;
- it addresses CPU allocation, contention, colocation or load prediction.

Papers on unrelated "scheduling" topics (traffic signals, project planning, networking PHY) were dropped.

1 **Verification.** Every paper below was checked against its DOI record in Crossref: title, authors, venue, volume and date. Abstracts came from Semantic Scholar where available.

2 **Reading depth.**

- For seven papers the summary is based on the published abstract.
- For three papers (marked † in the table) only the title and DOI metadata could be retrieved: the publisher pages are paywalled and no abstract was indexed. For those, the summary is limited to the scope their titles state, and no claim is made about their results.

Limitations. Full texts were not read, so findings come from abstracts. No Wiley paper passed screening. The sample is small (10 papers) and leans toward cloud/datacenter work, because that is where most 2023–2026 colocation research is published. Single-host Windows scheduling gets very little academic attention, which is itself a finding (see Gap 5).

---

2. Papers Reviewed

#	Reference	Venue (publisher)	Year
P1	J. Lin, Y. Chen, S. Gao, Y. Lu. *Fast Core Scheduling with Userspace Process Abstraction.* doi:10.1145/3694715.3695976	SOSP '24, ACM SIGOPS 30th Symposium on Operating Systems Principles (ACM)	2024

#	Reference	Venue (publisher)	Year
P2	R. Hossain, A. Banu, M. S. Arefin. *CPU scheduling from classical heuristics to learning-based and multicore paradigms: a systematic review, comparative analysis, and research roadmap.* Computing 108, art. 142. doi:10.1007/s00607-026-01736-8 †	Computing (Springer)	2026
P3	Z. Wang, S. Zhu, J. Li, W. Jiang, K. K. Ramakrishnan, M. Yan, X. Zhang, A. X. Liu. *DeepScaling: Autoscaling Microservices With Stable CPU Utilization for Large Scale Production Cloud Systems.* IEEE/ACM ToN 32(5). doi:10.1109/TNET.2024.3400953	IEEE/ACM Transactions on Networking (IEEE/ACM)	2024
P4	S. Choi, H. Eom. *Markov Chain-Based Resource-Efficient and QoS-Aware Scheduling for Latency-Critical and Best-Effort Tasks.* doi:10.1109/ACCESS.2025.3543964	IEEE Access (IEEE)	2025
P5	H. Zhao, W. Cui, Q. Chen, J. Leng, et al. *Improving Cluster Utilization Through Adaptive Resource Management for Deep Neural Network and CPU Jobs Colocation.* doi:10.1109/TC.2023.3303988	IEEE Transactions on Computers (IEEE)	2023
P6	D. Xu, F. Liu, B. Wang, X. Tang, Q. Wu. *Adaptive CPU sharing for co-located latency-critical JVM applications and batch jobs under dynamic workloads.* FGCS 180, 108387. doi:10.1016/j.future.2026.108387 †	Future Generation Computer Systems (Elsevier)	2026
P7	J. Li, W. Lin, H. Wu, J. Chen, D. Du, K. Li. *MixloadSched: An interference aware and energy efficient scheduling method for mixed workloads in cloud data centers.* FGCS 185, 108677. doi:10.1016/j.future.2026.108677 †	Future Generation Computer Systems (Elsevier)	2026
P8	T. Le Duc, C. Nguyen, P.-O. Östberg, et al. *Workload Prediction for Proactive Resource Allocation in Large-Scale Cloud-Edge Applications.* Electronics 14(16), 3333. doi:10.3390/electronics14163333	Electronics (MDPI)	2025
P9	S. Shao, X. Ding, B. Zhao, P. Ye, et al. *Attention-based workload prediction and dynamic resource allocation for heterogeneous computing environments.* doi:10.1038/s41598-026-38622-4	Scientific Reports (Nature Portfolio)	2026
P10	S. Tang, Y. Yu, H. Wang, G. Wang, W. Chen, Z. Xu, S. Guo, W. Gao. *A Survey on Scheduling Techniques in Computing and Network Convergence.* IEEE COMST 26(1). doi:10.1109/COMST.2023.3329027	IEEE Communications Surveys & Tutorials (IEEE)	2024

† Metadata only (title, authors, venue, DOI verified); abstract not retrieved.

2.1 Summaries

- **P1, Vessel (SOSP 2024).** Introduces *uProcess*, a userspace process abstraction that reschedules CPU cores between applications at sub-microsecond timescales without trapping into the kernel. The Vessel core scheduler colocates latency-critical and best-effort applications on time-shared cores with minimal switching cost. It depends on two emerging hardware features: userspace interrupts (Uintr) and memory protection keys (MPK).

Relevance: stock kernel scheduling is too coarse and too slow to protect latency-critical work from colocated batch work. The best fixes, however, need special hardware and a custom runtime.

- **P2, CPU scheduling systematic review (Computing 2026) †.** By its title, it reviews CPU scheduling from classical heuristics through learning-based and multicore approaches, and proposes a research roadmap.

Relevance: used only to place classical heuristic scheduling (static priorities and thresholds) as the baseline that newer work tries to move beyond. No specific findings are attributed to it.

- **P3, DeepScaling (IEEE/ACM ToN 2024).** Notes that operators set conservatively low, fixed CPU-utilization targets to meet SLOs, which wastes resources. DeepScaling *adaptively refines* the target CPU utilization and keeps it stable. It uses a spatio-temporal GNN to forecast workload, a DNN to estimate CPU utilization, and a modified DQN to choose the autoscaling policy.

Relevance: direct evidence that fixed CPU targets are a known weakness, and that forecasting load before acting beats reacting. It also shows the typical cost: three deep models trained offline at cluster scale.

- **P4, Markov-chain LC/BE scheduling (IEEE Access 2025).** Argues that resource-isolation methods for latency-critical (LC) tasks allocate resources reactively until QoS is met, and then hand everything left over to best-effort (BE) tasks. That ignores where BE performance saturates, and real-time adjustment is expensive. It proposes a weight-based Markov-chain model built from offline profiling, which decides allocations in advance.

Relevance: names two gaps directly: purely reactive adjustment, and the overhead of control actions.

- **P5, SODA (IEEE TC 2023).** Characterises how CPU-core allocation and CPU-side contention affect colocated DNN and CPU jobs. Its scheduler includes an *adaptive CPU allocator* and a *real-time contention eliminator*.

Relevance: contention from colocated work is a first-class problem, and both adaptive allocation and active contention removal are needed.

- **P6, Adaptive CPU sharing for LC JVM apps and batch jobs (FGCS 2026) †.** By its title, it studies adaptive CPU sharing between colocated latency-critical JVM services and batch jobs under dynamic workloads. That is the same problem shape as ThreadHaul's dispatch-API vs. batch-job scenario.

Relevance: confirms the problem is active in 2026. No results are attributed to it.

- **P7, MixloadSched (FGCS 2026) †.** By its title, an interference-aware, energy-efficient scheduler for mixed workloads in cloud data centres.

Relevance: interference beyond raw CPU share is a current research focus. No results are attributed to it.

- **P8, CDN workload prediction (Electronics 2025).** A prediction-plus-autoscaling framework combining S-ARIMA, LSTM, Bi-LSTM and OS-ELM in an adaptive feedback loop, evaluated on 18 months of production traces. It switches models to balance accuracy against compute cost.

Relevance: proactive, prediction-driven allocation outperforms reactive allocation. It also treats the forecaster's own cost as a design constraint.

- **P9, Attention-based prediction and allocation (Scientific Reports 2026).** Multi-head spatial-temporal attention forecasts workload, and allocation is framed as a multi-objective optimisation that explicitly accounts for prediction uncertainty. On cluster traces it reports 78.4% utilisation with 2.3% SLA violations.

Relevance: prediction is valuable, but the predictor must be designed around its own uncertainty.

- **P10, Scheduling in computing and network convergence (IEEE COMST 2024).** A survey of scheduling across cloud, edge and network resources treated as one pool.

Relevance: scheduling decisions increasingly span several machines, not a single node.

---

3. Research Gaps Identified

Gap 1: Static thresholds and purely reactive throttling

Classical schedulers and most production policies act on fixed thresholds after a limit has already been crossed (P2 baseline; P3's fixed utilization targets; P4's reactive isolation). By the time a static threshold has held for N samples, the latency-critical process has already been starved for N sampling periods. P3, P8 and P9 all show that forecasting load and acting *before* the limit is crossed works better.

Gap 2: Predictors too heavy for an on-host OS agent

The predictive systems in the sample are cluster-level controllers built on deep models: a GNN, DNN and DQN in P3, LSTM ensembles in P8, attention networks in P9. They need training data, GPUs or offline profiling (P4), and a separate control plane. None of them is suitable for a lightweight background service on the very server it

protects, where the monitor must not itself become a source of CPU contention. P4 and P8 explicitly treat control and prediction overhead as a cost.

Gap 3: CPU-only signals miss memory and I/O interference

Interference between colocated work is not only about CPU share (P5's CPU-side contention, P7's interference-aware scheduling, P1's colocation focus). Priority-class schedulers such as the Windows dispatcher only arbitrate CPU time. A latency-critical process can be starved by memory pressure or I/O contention even when its CPU share looks normal.

Gap 4: Coarse control and an all-or-nothing response

Fine-grained mechanisms such as P1's sub-microsecond core reallocation need special hardware (Uintr, MPK) and a custom userspace runtime, which a commodity Windows server lacks. With only OS priority classes, the response tends to be either too weak (a priority change that does not free enough CPU on a busy machine) or too blunt (stopping all batch work). P5's two-level design, an adaptive allocator plus a contention eliminator, suggests a graded response.

Gap 5: Single-node decisions in multi-node deployments

Scheduling research is moving toward decisions across many machines (P10, and P3 at cluster scale). Single-host monitors decide in isolation and cannot react to fleet-wide pressure. Separately, the sample contains almost no work on *Windows* single-host throttling, which suggests a practical gap between datacenter research and the commodity servers small logistics operators actually run.

---

4. How Each Gap Shaped the Design

Gap	Design decision in ThreadHaul-Daemon	Where
1. Static, reactive thresholds	**Predictive/adaptive thresholds.** A least-squares trend is fitted over the last `window_samples` pressure readings and forecast `horizon_samples` ahead. If load is rising and the forecast crosses the throttle threshold, the throttle and escalate thresholds are lowered by `slope × horizon`, capped at `max_adjust_percent`. Batch work is shed *before* starvation.	`Telemetry/AdaptiveThresholds.*`, config `adaptive.*`
1. (safety)	The recovery threshold never moves, and throttle always stays above recovery, so adaptation cannot cause oscillation or premature recovery (keeping the hysteresis band of P4's stability concern).	`AdaptiveThresholds::Offset`
2. Heavy predictors	**O(window) arithmetic, no model training.** The forecaster is a closed-form linear fit over about 10 numbers per second. **Self-monitoring** logs the daemon's own CPU and memory to prove it is lightweight: about 1% of one core and about 8 MB working set in our runs.	`Telemetry/ProcessMonitor.cpp` (`SelfMonitor`), log `self_monitor`
3. CPU-only signals	**Memory and I/O triggers.** System memory load, the critical process's working set and its I/O throughput are each expressed as a percentage of a configured limit, and fed through the same thresholds as CPU. The daemon acts on whichever signal is worst and logs which one triggered (`trigger_signal`).	`Daemon::Poll`, config `triggers.*`
3. (measurement)	**Explicit CPU normalisation** (`per_core` or `total`), so the same threshold means the same thing on a 4-core and a 64-core server.	`Telemetry/CpuMath.*`, config `telemetry.cpu_normalization`
4. Coarse/all-or-nothing control	**Graded escalation per batch process:** first demote to IDLE priority (SetPriorityClass); only if starvation continues, suspend every thread (SuspendThread); then resume and restore. Each batch process has its own state machine, so one can be suspended while another is only throttled.	`Actuator/StateMachine.*`

Gap	Design decision in ThreadHaul-Daemon	Where
5. Single-node decisions	**Central controller with cluster bias.** Each daemon streams heartbeats and actions to `ThreadHaulController`. When at least `--cluster-percent` of servers are hot, the controller sends every agent a bias that lowers their throttle thresholds. Servers not yet starved shed batch load early, which reduces fleet-wide pressure, and the bias is cleared when the cluster calms down.	`Daemon/Coordination.*`, `Controller/controller.cpp`
5. (commodity platform)	Built only on documented Win32 APIs available on any Windows Server. No special hardware (cf. P1) and no kernel driver.	whole project

5. Not Adopted, and Why

- **Learned predictors (DQN, LSTM, attention; P3, P8, P9).** They would break the lightweight requirement (Gap 2) and need training data we do not have for a new deployment. The adaptive module's interface (`AddSample` -> `Adjust`) is small enough that a learned model could replace the linear fit later.
- **Userspace core scheduling (P1).** Needs Uintr/MPK hardware and replacing the application runtime, both out of scope for a Windows service that manages unmodified processes.
- **Offline profiling (P4).** Logistics peaks are irregular, and the system must work on first install with no profiling period.

6. Conclusion

The 2023–2026 literature agrees that fixed, reactive thresholds waste capacity or protect latency-critical work too late. It also agrees that prediction helps, and that interference goes beyond CPU. Its solutions, however, mostly assume cluster-scale control planes, deep models or special hardware. ThreadHaul-Daemon takes the transferable ideas (prediction, multi-signal triggers, graded response, cluster-wide coordination) and puts them into a single lightweight Windows service that uses only standard APIs.